

Assignment 1 - Evaluation

Due date: 5.4.26

Intro

This assignment is a hands-on dive into LLM evaluation: from understanding the business use-case, through defining evaluation criteria, to performing evaluation using two methods - human evaluation and Judge Model (aka LLM-as-a-judge). Along the way you'll explore the strengths and trade-offs of each method, and grapple with the inherent challenge of evaluating outputs that have no single "correct" answer.

The business use case: generate product descriptions

The provided dataset contains e-commerce products with the following features: name, structured attributes, material, and warranty. Later you'll be asked to use a language model for crafting a persuasive, 50–90 word description of each product based on these features.

The evaluation criteria:

Criterion	Description	Rating
Fluency	Natural, easy-to-read sentences	good / ok / bad
Grammar	Correct spelling & punctuation	good / ok / bad
Tone	Matches friendly, credible sales voice	good / ok / bad
Length	50-90 words	good / ok / bad
Grounding	Sticks to information provided	good / ok / bad
Latency (avg. time per call)	Time to first byte / full response	good / ok / bad
Cost (avg. price per call)	Relative inference or API cost per 1K tokens	good / ok / bad

Task 1 - define your rubric (15 points)

Before generating or evaluating anything, you need a clear, repeatable scoring framework. Define it now so that every evaluation decision — whether made by you or by a judge model later — follows the same rules.

1. **Criterion definitions** - For each criterion in the table above, write explicit definitions for *good*, *ok*, and *bad*. The goal is to minimize subjectivity: another person reading your rubric should reach the same verdict you would. Example for Length: good = 50–90 words, ok = 40–49 or 91–110 words, bad = outside that range.
2. **Pass / fail definition:**
 - a. **Cumulative pass bar** - Define the minimum combination of ratings a description must achieve to pass. For example: "at least three good ratings and zero bad ratings."
 - b. **Go / no-go rules** - Define any single criterion that triggers automatic failure regardless of other scores. For example: "if Grounding is not good, the description is rejected."

Task 2 - generate the description for every product (20 points)

Prompt

Using the rubric you defined, now write a system prompt and use it to generate a description for every product in the dataset. Recall that the prompt should instruct the model to produce a persuasive 50–90 word product description based on the provided information (name, structured attributes, material, warranty). Apply the prompting guidelines covered in class.

Model

Choose one of the following models from [Nebius Token Factory](#) and use it for generating the

descriptions:

1. Gemma-2-9b-it
2. Meta-Llama-3.1-8B-Instruct

Structured Output

For each API call, collect the following fields into a dictionary:

- generated_description
- latency_ms - end-to-end generation time in milliseconds
- input_tokens - number of tokens sent to the model (including the system prompt)
- output_tokens - number of tokens received from the model

Storage

Collect all results into a DataFrame. Add blank columns for each rubric criterion and a blank final_score column (pass / fail). Save the DataFrame as an Excel file named `assignment_01.xlsx`.

Note: the models above are relatively small, and you'll probably get some bad results. That's actually good since it will give you interesting results to evaluate and improve later on. You won't lose points for bad descriptions as long as all the rest is correct.



Task 3 - manual (human) evaluation (10 points)

Open the assignment_01 sheet you created in task 2:

1. **Cost calculation** - Add a cost column. For each row, convert `input_tokens` and `output_tokens` into cost in USD using the pricing for your chosen model (note: input and output tokens are priced differently).
2. **Rate each criterion** - Choose 10-15 products and for each of them rate each criterion as good / ok / bad according to the rubric you defined in Task 1.
3. **Final**

score - Apply your cumulative pass bar and go/no-go rules from Task 1 to set the **final_score** for each row.

4. **Baseline analysis** - Review your scores across all products. Which criteria performed best? Which performed worst? Use this analysis to guide your improvement strategy in the next task.



Task 4 - improvement cycle (15 points)

Now that you've established a baseline score, iterate to achieve better results.

Ideas to explore (you don't have to try all of them!)

- **Prompt engineering** – rewrite the system prompt, add/change few-shot examples, or enforce stricter constraints.
- **Model choice** – try a different model from the token factory - bigger one (e.g., ~30B), or different architecture..
- **Decoding parameters** - adjust temperature, top_p, top_k, or max_new_tokens to balance creativity vs. factuality.
- **Post-processing** – run grammar-checking or length trimming after generation.
- **Ensembling** – combine outputs from two models.

For each experiment, document:

1. **What you changed**
2. **Why you expected it to help**
3. **New evaluation scores (re-evaluate using your Task 1 rubric)**



Task 5 - create a judge model (20 points):

In the previous tasks you manually evaluated each generated description - thorough but slow and hard to scale. In this task you'll build an automated judge: an LLM that grades descriptions using the same rubric you defined in Task 1.

For this you'll need to choose the model, write a judge prompt and define the output schema.

Model - Start with the model you **did not** use in Task 2. If you find it struggles as a judge, you may switch to a larger model (e.g., Qwen3-30B-A3B-Instruct-2507). Document why you made the switch.

Prompt - Write a judge prompt that includes your rubric definitions so the model applies the same standards you used during manual evaluation. Exclude cost and latency criteria - those are measured programmatically. Think carefully about what context the judge needs to evaluate each criterion - especially **Grounding**.

Output - for each criterion the judge should return:

- explanation (string) — reasoning for the verdict
- verdict (enum: good / ok / bad)

Note that explanation comes before verdict in the schema. In your submission, explain why this ordering matters.

Use a [Pydantic schema](#) to enforce this structure via the API's structured output support.

[Here](#) you'll find a short explanation as to why using Pydantic.



Task 6 - run and analyze the judge (20 points):

1. **Sanity check** - run the judge on 5 products. Review the explanations and verdicts manually — do they make sense? Does the judge apply your rubric correctly? Adjust the prompt if needed before proceeding.
2. **Full run** - run the judge on all products. Compute the `final_score` for each product

using the same pass bar and go/no-go rules from Task 1. Store everything back into the spreadsheet.

3. **Compare to human evaluation** - for each criterion, compare the judge's verdicts against your manual scores from Task 3. Compute an agreement rate per criterion. Where do they agree? Where do they diverge? Try to explain why.
4. **Criterion-by-criterion judging** - instead of asking the judge to evaluate all criteria at once, run it separately for each criterion (one call per criterion per product). Does this change the results? Why might isolating criteria lead to different outcomes? Compare agreement with your human scores — did it improve?
5. **Analysis** - reflect on what you found:
 - a. What are the practical trade-offs between human evaluation and LLM-as-a-judge (think: cost, scale, consistency, accuracy)?
 - b. Which approach would you recommend for a production system that generates thousands of descriptions daily?